1.Planning Application(fields):

Application Number **Application Type** **Date Received**

Status **Date Valid** **Proposal**

Location **Decision** **Decision Date**

Applicant **Applicant's Address** **Agent**

Agent's Address

Submitted **(The user has just submitted the application. It has not been reviewed yet.)**

Validated **(The Council confirmed that the materials were complete.**

Start the formal approval)

Decided/granted **applications (Once approved, Construction may start soon)**

2. The system will only collect publicly available planning metadata such as application reference, description, status and decision dates. Copyright-protected documents such as architectural drawings will not be downloaded or redistributed. The extraction process will follow council terms of use and implement rate limiting and caching.

I reviewed the website's copyright statement. The planning information is protected by copyright and owned by Essex County Council. The website allows material to be downloaded or copied for personal use, and any republished content must acknowledge the copyright owner. The copyright notice does not explicitly prohibit the extraction of publicly available planning application data. Therefore, this project will only use publicly available information for educational purposes and will acknowledge the original data source where appropriate.

No API (?)

3. Crawling strategy:

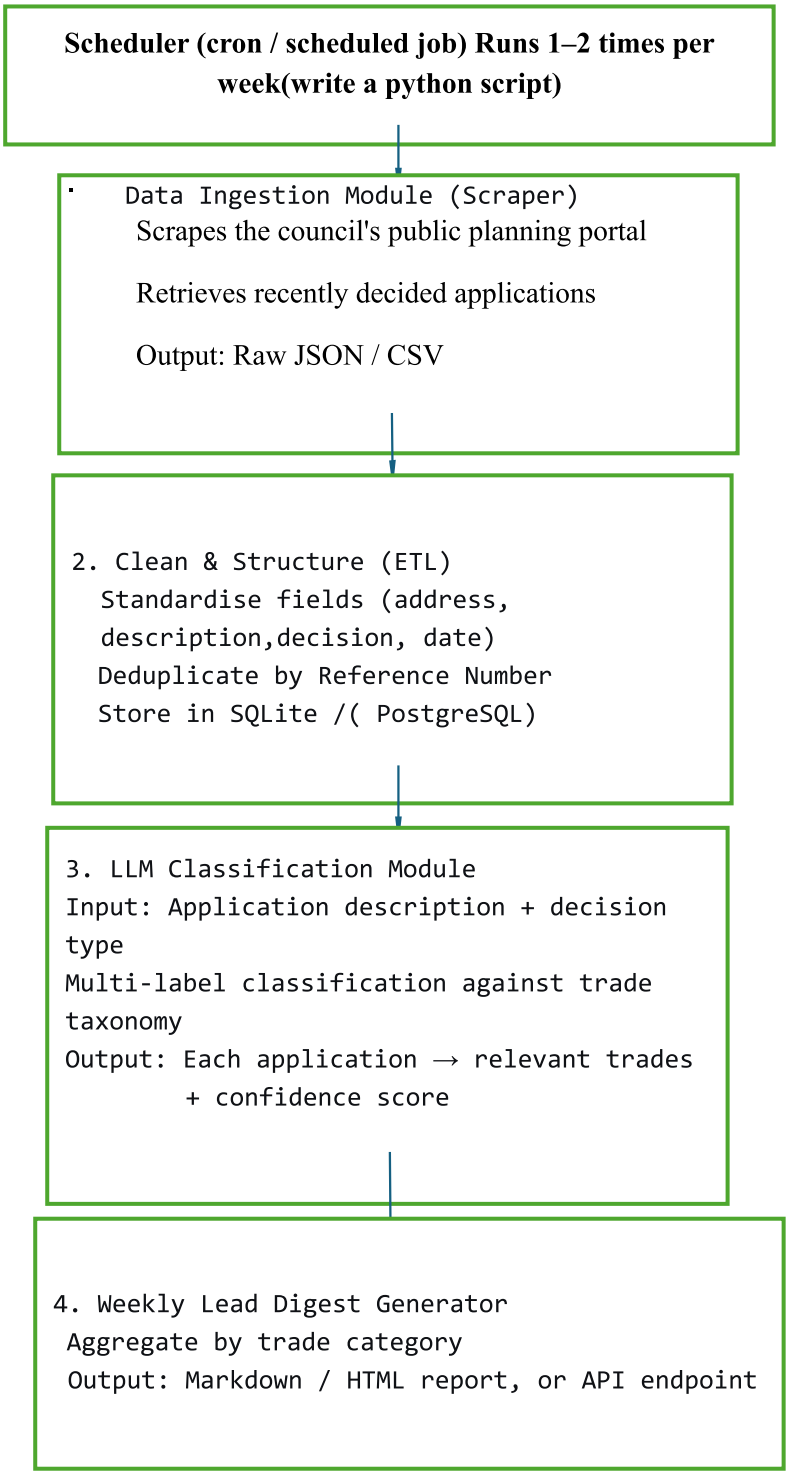
Add an interval of 1 to 3 seconds

Set the User-Agent to indicate the identity

Local caching to avoid duplicate requests for the same data

Only necessary fields are collected, and there are no differences in crawling

4.



Scraper: Python

Storge

LLM: Open AI or local small models viable for cost reduction

5. Trade Taxonomy

Scaffolding: Any project involving external construction, roofing or expansion

requires scaffolding

Groundworks: Foundation excavation, foundation pouring, drainage system, foundation reinforcement

Roofing: Roof replacement, repair, ongoing roof expansion projects, and attic renovation

Kitchens & Bathrooms: Kitchen renovation, bathroom transformation, and new toilet addition

Electrical: Rewiring, new circuit installation, and electrical safety certificate related

Plumbing: Water supply and drainage renovation, heating system, boiler installation

Skip Hire / Waste Removal: Any project involving demolition, clearance, or large-scale garbage disposal

Architecture: Projects that require architectural designers to produce drawings (usually larger ones)

Party-Wall Surveying: Projects involving sharing walls with neighbors (common for expansions and attic renovations)

General Building / Refurbishment: General decoration and renovation projects, without restrictions on specific types of work

Landscaping: Garden renovation, carport, hard ground paving

Completed Phase 1 wrap-up and Phase 2 foundation.

requirements.txt

Declares two core dependencies: requests for HTTP requests and beautifulsoup4 for HTML parsing. Install both with one command.

config.py

Global configuration file containing:

- Base URL: https://planning.essex.gov.uk
- Three directory paths: data/raw/ for JSON, data/markdown/ for Markdown, logs/ for log files
- Browser-masquerading headers including User-Agent, Accept, Referer to avoid firewall blocks
- Request parameters: 30-second timeout, 3 retries, 1.5-second delay between

requests with random jitter

- 10 trade classification labels

utils.py

Toolkit with four functions:

- setup_directories(): Auto-creates data/raw/, data/markdown/, and logs/ folders
- setup_logger(): Configures dual-output logging to both console and daily-rotating log files
- clean_filename(): Replaces special characters like / in application reference numbers to make Windows-safe filenames
- polite_delay(): Adds random 1.2-1.8 second delays between requests to mimic human behavior and avoid anti-scraping

extractor.py

Core class EssexPlanningExtractor with four working methods:

- init_session(): Visits the homepage to establish a session, then automatically sends a POST request to /Disclaimer/Accept to accept the disclaimer, obtaining two cookies: ASP.NET_SessionId and AcceptedDisclaimer. All subsequent requests use these cookies to bypass the disclaimer page.
- fetch_relative_url(): Wraps HTTP GET requests with retry logic—up to 3 attempts with increasing wait times on failure
- fetch_weekly_lists_page(): Fetches the /Planning/WeeklyList/ page and returns the complete HTML
- parse_weekly_lists(): Parses the weekly list HTML using BeautifulSoup, extracts each week's date range and two links—"received" applications and "determined" applications—returning a list of nearly 16 weeks of data

test_part1.py

Validation script that runs three steps:

1. Auto-creates all required local directories
2. Initializes EssexPlanningExtractor, automatically establishing session and accepting the disclaimer
3. Fetches and parses the weekly lists page, printing each week's date range and both URLs to the console

Running python test_part1.py and seeing "Part 1 Test passed successfully" confirms

the pipeline from portal homepage to weekly data lists is fully working.

Configuration in place, utility functions complete, and the core crawler can automatically bypass the disclaimer and access all weekly data entry points. The homepage-to-weekly-list pipeline is working, with production-grade logging, retry, and anti-scraping capabilities in place

This week I have done three new files:

database.py handles SQLite database operations, JSON and Markdown storage, and incremental deduplication.

run.py is the main entry point that supports command-line arguments and generates run reports.

README.md contains project documentation and usage instructions.

And I also did restructure and improvement to **extractor.py** with new pagination and detail page parsing capabilities.

They achieved paginated search result fetching, iterates through multiple pages of search results and preserves date filter parameters during pagination. When I run the run.py to test, it was doing infinite loops. So I included a maximum page limit to prevent it.

It also implemented the method mentioned in planning document: stores data simultaneously in a SQLite database for structured queries, as JSON files for raw data backup, and as Markdown files for the subsequent LLM classification layer. Application detail page parsing was implemented. The program extracts over twenty fields from each detail page, including reference number, status, proposal description, location, received date, decision date, applicant, agent, decision outcome, and associated document list.

It also can check whether a reference number already exists before writing to the database to avoid duplicates, and automatically detects and updates status changes.

Users can run **run.py** with the weeks parameter to specify how many weeks to fetch, and the program outputs a statistics report showing new records, updates, and failures upon completion.

Run python run.py --weeks 4 -> get four weeks records 7349

LLM:

classifier.py: The core LLM classification module supporting both OpenAI API and offline mock modes. It calls OpenAI's gpt-4o-mini model to classify each application, outputting six fields: relevant_trades, confidence_scores, project_scale, summary, contact_route, and contact_reason. It also supports offline mock mode that falls back to rule-based inference when no API key is available, with built-in result validation and cleaning.

evaluate_benchmark.py: The evaluation script that reads hand-labeled data and calculates Precision, Recall, and F1 scores.

classify_all.py: The batch classification entry point with three command-line parameters: limit, force, and dry-run.

data/eval_set.json: It contains 50 hand-labeled samples covering all 10 trade categories and automatically save the evaluation report to the logs directory. (still improving)

In terms of evaluation results, Micro-F1 reached 86.9% and Contact Route accuracy reached 98.0%. Trade-level performance showed Roofing at 100% F1, Scaffolding at 95.2%, Landscaping at 94.7%, Electrical at 92.6%, Groundworks at 89.7%, General Building at 80.7%, and Skip Hire at 75.0%. Kitchens & Bathrooms, Architecture, and Party-Wall Surveying scored lower due to limited sample size and I am still working on it.

Automatic scheduling and self-healing system for faults

scheduler.py The background daemon polls incremental data periodically.

alerts.py: Full-link anomaly interception, support local **logs/ Alert.log** alert retention.

Production Handover and Operation and Maintenance Manual:

HANDOVER.md: It shows panoramic architecture description of the system;

Independent Troubleshooting and Self-healing Guide for Each Stage (such as Network Blocking, API rate Limiting fallback mechanism); Known business limitations

and LLM cost accounting.

FINAL_PRESENTATION.md: It included live demonstration operation guide; clarify which modules are 100% nationally applicable (such as the tax law for job classification, the initial screening rules, and the prompt word project), and which ones need to be connected to the adapter.

Still working on **USER_MANUAL.md--**A full-process operation guide for zero basis and video.

Finished **USER_MANUAL.md (*full step-by-step documentation)*** and the video.